

IPre Sueño-Vigilia y Compositional Learning

(IMT2985)

Kevin Cardemil Ulloa
kevin.cardemil@uc.cl

Contenidos

I	Semana 0	3
1.1.	Introducción y Definiciones Básicas	3
	Lenguaje Formal	3
	Entropía de Shannon (Teoría de la Información)	3
	Principio de Longitud de Descripción Mínima (MDL)	3
	Encoders y Autoencoders	3
1.2.	Reinforcement Learning	3
	Una Introducción al Reinforcement Learning	3
	Elementos del Reinforcement Learning	3
	Procesos de Decisión de Markov Finitos	4
1.3.	DreamerV3	5
1.4.	Notas de Composicionalidad	5
	¿Qué no entendí?	5
1.5.	Computabilidad	6
	Máquinas de Turing	6
1.6.	Referencias Semana 0	7
II	Semana 1	8
2.1.	Complejidad de Kolmogorov	8
	Teoría de la Información	8
2.2.	MDL	8
2.3.	Cálculo Lambda	8
2.4.	DreamCoder	8
	Domain-Specific Lenguajes (DSL)	8
	Síntesis Inductiva de Programas	8
	Typed Grammars	8
	Connection to Helmholtz Machine and VAEs	8
	Resultados Experimentales	8
2.5.	Flat Minima	8
	Resultados Experimentales	8
2.6.	Glosario de la Semana	8
	Weight Decay	8
	Hinton & van Camp	8
	Optimal Brain Surgeon / Damage (OBS / OBD)	8
	Algoritmo de Gibbs	8
	Distancia de Kullback-Leibler	8

I Semana 0

1.1. Introducción y Definiciones Básicas

Lenguaje Formal

Entropía de Shannon (Teoría de la Información)

Principio de Longitud de Descripción Mínima (MDL)

Encoders y Autoencoders

VAEs

1.2. Reinforcement Learning

Esta sección está basada en los capítulos 1 y 3 del libro *Reinforcement Learning: An Introduction* de Richard S. Sutton y Andrew G. Barto (2014-2015).

Una Introducción al Reinforcement Learning

El **Reinforcement Learning** (RL) es un acercamiento *computacional* al *aprendizaje por interacción*, el cual pareciera subyacer a cualquier teoría del aprendizaje e inteligencia. La guía de este tipo de aprendizaje son las metas o retornos que, por lo general, son a largo plazo.

Por un lado, es diferente al **Aprendizaje Supervisado** (*Supervised Learning*), el cual consiste en aprender, a partir de un conjunto de entrenamiento etiquetado, la manera de clasificar o regresionar los nuevos ejemplos, sin embargo, no se basa en la interacción con el ambiente, como sí lo hace el RL.

Por otro lado, el RL es diferente al **Aprendizaje No Supervisado** (*Unsupervised Learning*), el cual consiste —en esencia— en encontrar patrones o *estructura* en un conjunto de entrenamiento sin etiquetas, no obstante, el RL no busca una estructura, sino que maximizar el retorno.

De esta manera, este modelo de aprendizaje lo asumiremos como un tercer paradigma del **Aprendizaje de Máquina** (*Machine Learning*).

Un reto que aparece en el RL es el tradeoff entre *explorar* y *explotar*: explorar para realizar mejores acciones a futuro; explotar decisiones que se sabe que son provechosas.

Finalmente, el RL es parte de una tendencia en la IA de retorno a principios generales y simples. Los métodos basados en principios generales se conocen como **métodos débiles** («*weak methods*»), mientras que aquellos basados en conocimiento específico se denominan **métodos fuertes** («*strong methods*»).

Elementos del Reinforcement Learning

El ciclo de aprendizaje en el RL tiene dos actores principales: el **agente** y el **ambiente**. En esencia, el agente toma decisiones buscando una manera de actuar dado cierto estado y el ambiente, el objeto con el que el agente interactúa, se encarga de indicarle una recompensa dado su actuar y el siguiente estado en que estará.

De manera más técnica, un sistema de RL se compone de: una *política*, una *señal de recompensa*, una *función de valor* y, opcionalmente, un *modelo del ambiente*.

Una **política** (*policy*) es una función que el agente aprenderá y que define la manera en que el learner se comporta en determinado instante. Corresponde a una especie de reglas estímulo-respuesta. En general, estas funciones son estocásticas.

Una **señal de recompensa** (*reward signal*) es el *reward* que entrega el ambiente al agente en cada paso de tiempo. Ella afecta directamente la manera en que el agente se comportará. Por lo general, el objetivo del agente es *maximizar la recompensa total*, a largo plazo.

La **función de valor** (*value function*) especifica qué es lo bueno a largo plazo, indicando el monto total que puede esperar el agente a largo plazo. Es justamente parte de lo que queremos aprender.

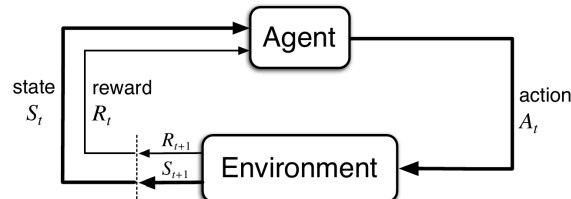
El **modelo del ambiente** es un objeto que imita el comportamiento del ambiente y que permite realizar inferencias de cómo el ambiente se comportará. En el RL podemos encontrar métodos **basados en modelos** (*model-based*), que son justamente aquellos que tienen un modelo del ambiente, y otros métodos más simples, denominados métodos **libres de modelo** (*model-free*).

Procesos de Decisión de Markov Finitos

Este problema define el campo del RL.

Una especificación completa de un ambiente define una **tarea**, esto es, una instancia de RL.

A continuación se empieza a introducir notación. El agente y el ambiente interactúan en instantes de tiempo discretos, $t = 0, 1, 2, \dots$ (aunque existen ideas que consideran el tiempo continuo). En cada instante, el agente recibe una **representación del estado del ambiente**, $S_t \in \mathcal{S}$, donde $\mathcal{S}$ es el **espacio de estados** (*state space*). En esa situación, escoge una **acción** $A_t \in \mathcal{A}(S_t)$, donde $\mathcal{A}(S_t)$ corresponde al conjunto de acciones posibles dado el estado S_t . Como consecuencia, un instante de tiempo después, el ambiente retorna una **recompensa** $R_{t+1} \in \mathcal{R}$, donde $\mathcal{R} \subset \mathbb{R}$ es un conjunto de posibles recompensas. Además, el ambiente le indica el nuevo estado S_{t+1} al agente.



La política del agente la denotamos como $\pi_t(a|s)$, entendida como la probabilidad de que $A_t = a$ dado el estado $S_t = s$. Así, los métodos de RL consistirán en especificar cómo el agente modifica su política a partir de los resultados que obtiene.

Nota 1.2.1. La frontera entre agente y ambiente suele estar más cerca al agente de lo que se cree. Por ejemplo, los motores y sensores de un robot se consideran parte del ambiente (pues pueden entenderse como estados y recompensas), no del agente. Normalmente, basta con preguntarse si el agente *no* puede cambiar algo arbitrariamente. De ser así, entonces ese algo no pertenece al agente. De esta manera, la frontera agente-ambiente corresponde al límite de control del agente, no de su conocimiento. ■

1.3. DreamerV3

1.4. Notas de Composicionalidad

¿Qué no entendí?

1.5. Computabilidad

Máquinas de Turing

1.6. Referencias Semana 0

- **S0 - Notas-ApprComposicional.** (Una pasada para ver qué no conozco, otra para entender todo.)
- Paper DreamCoder: **Mastering Diverse Domains through World Models** (Sólo hasta Sección 2).
- **Li, M. & Vitányi, P. (2008). An Introduction to Kolmogorov Complexity and Its Applications**, 3ª ed. Springer. (C1 y C2)
- **Sipser, M. (2012). Introduction to the Theory of Computation**, 3ª ed. — Capítulos 3 (máquinas de Turing) y 4 (decidibilidad, problema de la parada).
- **Grünwald, P. D. (2007). The Minimum Description Length Principle.** MIT Press (C1)

II Semana 1

2.1. Complejidad de Kolmogorov

Teoría de la Información

2.2. MDL

2.3. Cálculo Lambda

2.4. DreamCoder

Domain-Specific Languages (DSL)

Síntesis Inductiva de Programas

$$\{\ell_{x_i}\}$$

Typed Grammars

Definición 2.4.1. (Expression tree).



Connection to Helmholtz Machine and VAEs

Helmholtz Machine

Variational Inference Primer

Diferencias y similitudes

Resultados Experimentales

2.5. Flat Minima

Resultados Experimentales

2.6. Glosario de la Semana

Weight Decay

Hinton & van Camp

Optimal Brain Surgeon / Damage (OBS / OBD)

Algoritmo de Gibbs

Distancia de Kullback-Leibler